



# Подсистема CRC Python IDE для анализа и ассистирования модификаций кода

*Кузнецов Илья Александрович*

Выпускная квалификационная работа

Уровень образования: магистратура

Направление 09.04.04 «Программная инженерия»

Основная образовательная программа ВМ.5666.2023 «Программная инженерия»

Научный руководитель:

профессор кафедры системного программирования, д. т. н. Кознов Д. В.

Консультант:

ведущий инженер ключевых проектов ООО «Техкомпания Хуавэй», Тропин Н. В.

Рецензент:

ведущий инженер ключевых проектов ООО «Техкомпания Хуавэй», к. ф.-м. н. Волков М. В.

Санкт-Петербургский государственный университет

# Введение

- ❑ Python — один из наиболее востребованных языков для серверных систем, автоматизации, анализа данных и ИИ-решений
- ❑ Сложность проектов на Python требует не только проверки синтаксиса, но и средств анализа, исправления и контекстной поддержки при разработке
- ❑ По данным Stack Overflow Developer Survey, 84% разработчиков уже используют или планируют использовать ИИ-инструменты, а 51% применяют их ежедневно
- ❑ ИИ-агенты повышают требования к точности семантического контекста и управляемости изменений
- ❑ CRC Python IDE нуждается в целостной подсистеме для анализа и ассистирования модификаций кода

# Цель и задачи

## Цель

Проектирование и реализация подсистемы для среды разработки CRC Python IDE, которая должна предоставлять инструменты (Features) для анализа и ассистирования модификаций Python-кода

## Задачи

- ❑ Провести обзор инструментов для анализа и ассистирования модификаций кода в существующих средствах разработки для языка Python
- ❑ Выполнить проектирование подсистемы, определив группы инструментов, необходимых пользователям, а также улучшения модели кода CRC Python IDE, необходимые для реализации этих инструментов
- ❑ Выполнить реализацию спроектированной подсистемы и интегрировать ее в CRC Python IDE
- ❑ Провести всестороннее тестирование разработанной подсистемы, модернизировав существующую тестовую инфраструктуру в проекте CRC Python IDE
- ❑ Провести апробацию и доработку подсистемы на основе обратной связи от пользователей CRC Python IDE

# Обзор: статические анализаторы, среды разработки (IDE) и системы на основе ИИ-агентов (ADE)

- ❑ Статические анализаторы
  - ❑ Муру — классический анализатор типов, выявляет несоответствия между ожидаемыми и фактическими типами
  - ❑ Pyright — быстрый анализатор для больших кодовых баз, интегрирован в Pylance для интерактивных сценариев
  - ❑ Pyrefly — совмещает быстрый типовой анализ с языковым сервером и базовыми возможностями IDE
- ❑ Среды разработки (IDE)
  - ❑ PyCharm — развитая интеграция инспекций, рефакторинга и навигации
  - ❑ VSCode с Pylance — модульная архитектура через Language Server Protocol (LSP), объединяет анализ, исправления и рефакторинг
  - ❑ CRC Python IDE — корпоративная среда на базе VSCode с собственной моделью кода и внутренними сервисами
- ❑ Системы на основе ИИ-агентов (ADE)
  - ❑ AI Assistant — встроенный ИИ-агент Junie для автономного планирования и исполнения действий через Model Context Protocol (MCP)
  - ❑ Copilot — диалоговые и агентные сценарии в VSCode с режимами Ask, Edit и Agent
  - ❑ Cursor — глубокая интеграция агентных сценариев с изолированным окружением для автономной работы

# Обзор: сравнение инструментов

| Класс решений                      | Статический анализ                                                                                                                                  | Модификация кода                                                                                                                                   | Ассистирование                                                                                                                            |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Статические анализаторы            | Сильная сторона — раннее выявление ошибок и несоответствий в типах, ссылках и использовании конструкций языка                                       | Как правило, ограничиваются сообщением о проблеме, а изменения в коде не являются их основным предназначением                                      | Ассистирование присутствует косвенно: результаты анализа могут использоваться другими инструментами                                       |
| Среды разработки (IDE)             | Статический анализ встроен в редактор и связан с навигацией, областями видимости и контекстом проекта                                               | Поддерживаются корректирующие модификации и рефакторинг с предпросмотром изменений                                                                 | Предоставляются контекстные подсказки, работающие поверх различных инструментов и сервисов                                                |
| Системы на основе ИИ-агентов (ADE) | Статический анализ используется в качестве структурированной контекстной информации, качество которой зависит от интеграции с другими инструментами | Возможны многошаговые и многофайловые модификации, однако надежность таких сценариев ограничена без дополнительной верификации семантики программы | Сильная сторона — диалоговые и агентные сценарии, но есть риск потери актуальности контекста без структурированного доступа к модели кода |

# Архитектура: инструменты подсистемы

| Выявленная особенность решений                                                                                                                    | Запрос Python-разработчиков                                                                                              | Следствие для проектирования                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Статический анализ надежно решает задачи проверки типов, ссылок, сигнатур и корректности использования конструкций Python                         | Находить проблемы в коде до выполнения программы и получать детальное объяснение их причины                              | Требуется поддержка статического анализа кода, способного выявлять проблемы до выполнения программы и объяснять их причины                                                                        |
| Среды разработки связывают проблемы в коде с корректирующими модификациями в точке возникновения проблемы                                         | После обнаружения проблемы получать применимое исправление без ручного изменения кода                                    | Требуется поддержка контекстно-зависимых исправлений, применяемых в точке возникновения проблемы                                                                                                  |
| Рефакторинг в средах разработки является самостоятельным классом эквивалентных преобразований кода с предпросмотром и контролем области изменений | Выполнять более общие преобразования кода, улучшающие структуру программы и поддерживаемость проекта                     | Требуется поддержка рефакторинга кода, управляемого на уровне семантики программы                                                                                                                 |
| Подсказки превращают результаты анализа в контекстно-зависимую информацию, доступную во время редактирования                                      | Быстро получать сведения о декларациях, типах, сигнатурах и использовании объектов непосредственно в редакторе           | Требуется поддержка информационного ассистирования для пользователей, предоставляющего контекстные сведения о декларациях, типах, сигнатурах и использовании объектов непосредственно в редакторе |
| ИИ-агенты способны выполнять комплексные сценарии модификации кода, но без структурированного семантического контекста их надежность ограничена   | Использовать ИИ-агентов при работе с кодовой базой с минимальными потерями точности контекста и управляемостью изменений | Требуется поддержка информационного ассистирования для ИИ-агентов в виде структурированного семантического доступа к сведениям о коде и безопасным операциям по его модификации                   |

# Архитектура: улучшения модели кода

- ❑ PSI<sup>1</sup> — расширение и стабилизация синтаксического и семантического представления программы
- ❑ Resolve<sup>2</sup> — уточнение и конфигурирование возможностей разрешения ссылок
- ❑ Type Inference<sup>3,4</sup> — доработка интерфейса вывода типов

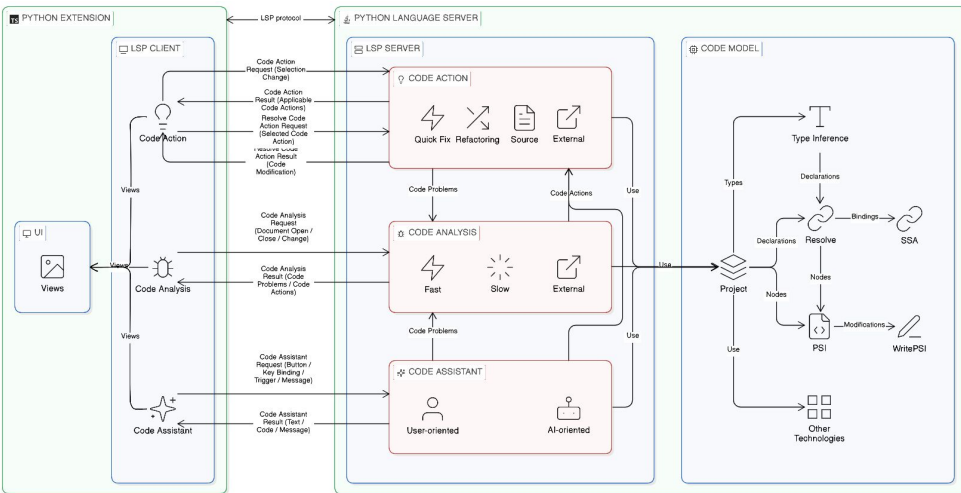
1 — Божнюк А. // Система модификации структуры исходного кода для интегрированных сред разработки // 2023 // СПбГУ

2 — Степанов С. // Потокочувствительный анализ программ на языке Python // 2023 // ИТМО

3 — Мирошников В. // Статический вывод типов для языка Python в интегрированной среде разработки // 2023 // СПбГУ

4 — Мирошников В. // Инкрементальный статический вывод типов для языка Python // 2025 // СПбГУ

# Архитектура: структура подсистемы





## Реализация: инструменты для статического анализа кода

- ❑ Для статического анализа кода было реализовано 40 инструментов
- ❑ Репрезентативные примеры: Syntax Correctness, Reference Resolve, Expected Type, Unreachable Code и др.



## Реализация: инструменты для исправления кода

- ❑ Для исправления кода было реализовано 12 инструментов
- ❑ Репрезентативные примеры: Install Package, Import Reference, Create Reference, Add Qualifier и др.



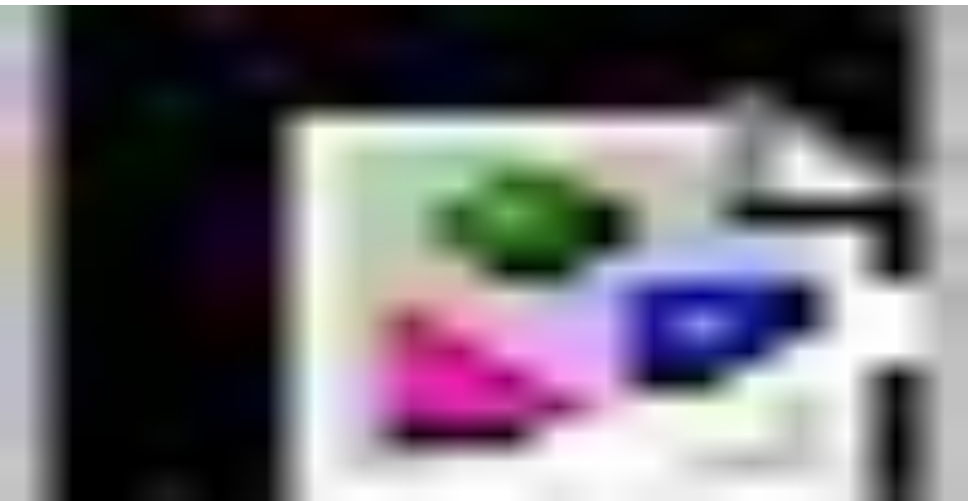
## Реализация: инструменты для рефакторинга кода

- ❑ Для рефакторинга кода было реализовано 8 инструментов
- ❑ Репрезентативные примеры: Extract, Inline, Rename, Optimize Imports и др.



## Реализация: информационные ассистенты для пользователей

- ❑ Для информационного ассистирования пользователей было реализовано 3 инструмента
- ❑ Репрезентативные примеры: Hover, Signature Help и Inlay Hints



## Реализация: информационные ассистенты для ИИ-агентов

- ❑ Для информационного ассистирования ИИ-агентов было реализовано 4 инструмента
- ❑ Репрезентативные примеры: Problems MCP, Hover MCP, Signature Help MCP и Rename MCP



# Тестирование: модульное, регрессионное, комплексное

- ❑ Модернизация тестовой инфраструктуры
  - ❑ Единая система с конфигурируемыми тестовыми проектами
  - ❑ Фиксация ожидаемых результатов для всех подгрупп инструментов
  - ❑ Поддержка маркеров позиции курсора и выделенных диапазонов
- ❑ Модульное тестирование
  - ❑ 3000 тестовых сценариев
  - ❑ Покрытие всех подгрупп инструментов
  - ❑ Проверка пограничных случаев
- ❑ Регрессионное тестирование
  - ❑ 5 бенчмарков, по одному для каждой подгруппы инструментов
  - ❑ 30 выборочных Python-проектов (Django, Numpy, Transformers и др.)
  - ❑ Мониторинг в Grafana с сохранением метрик в Victoria Metrics
- ❑ Комплексное тестирование
  - ❑ Автотесты на основе фреймворка Selenium
  - ❑ Проверка взаимодействия редактора, языкового сервера и UI
  - ❑ Воспроизводимые пользовательские сценарии

# Тестирование: мониторинг

- ❑ Визуализация результатов бенчмарков в системе мониторинга Grafana
- ❑ Сохранение метрик времени выполнения анализа, скорости исправлений и латентности ассистентов в базе данных Victoria Metrics



# Апробация

- ❑ Апробация подсистемы была проведена на основе отзывов пользователей CRC Python IDE, по их запросу было реализовано 3 новых инструмента и исправлено 10 проблем
  - ❑ Version Compatibility, Add Superclass Constructor Call и Optimize Imports
  - ❑ Ложно-положительные/отрицательные срабатывания в анализах кода, некорректные/неэквивалентные модификации кода и нерелевантные подсказки от ассистентов



# Заключение

## Результаты

- ❑ Рассмотрены инструменты для анализа и ассистирования модификаций кода в статических анализаторах Мypy, Pyright и Pyrefly, а также в средах разработки (IDE) PyCharm, VSCode и CRC Python IDE и, наконец, в системах на основе ИИ-агентов (ADE) AI Assistant, Copilot и Cursor; определены ключевые особенности этих решений и соответствующие им запросы Python-разработчиков
- ❑ Спроектирована подсистема среды разработки, включающая 3 следующие группы инструментов: (i) статический анализ, (ii) исправление и рефакторинг кода, а также (iii) информационные ассистенты для пользователей и ИИ-агентов; спроектированы необходимые улучшения модели кода CRC Python IDE: расширение и стабилизация синтаксического и семантического представления программы (PSI), уточнение и конфигурирование возможностей по разрешению ссылок (Resolve), доработка интерфейса вывода типов (Type Inference)

# Заключение

- ❑ Выполнена реализация спроектированной подсистемы
  - ❑ реализованы улучшения в модели кода CRC Python IDE
  - ❑ реализовано 40 инструментов для статического анализа кода (Syntax Correctness, Reference Resolve, Expected Type, Unreachable Code и др.)
  - ❑ реализовано 12 инструментов для исправления кода (Install Package, Import Reference, Create Reference, Add Qualifier и др.)
  - ❑ реализовано 8 инструментов для рефакторинга кода (Extract, Inline, Rename, Optimize Imports и др.)
  - ❑ реализовано 3 информационных ассистента для пользователей (Hover, Signature Help и Inlay Hints)
  - ❑ реализовано 4 информационных ассистента для ИИ-агентов (Problems MCP, Hover MCP, Signature Help MCP и Rename MCP)

# Заключение

- ❑ Выполнена модернизация тестовой инфраструктуры CRC Python IDE (добавлена поддержка конфигурируемых тестовых проектов с фиксацией ожидаемых результатов для каждой группы инструментов), создано 3000 модульных тестов и 5 бенчмарков для регрессионного тестирования подсистемы, а в качестве комплексного тестирования настроены автотесты с помощью фреймворка Selenium
- ❑ Проведена апробация подсистемы на основе отзывов разработчиков и пользователей CRC Python IDE, что способствовало созданию 3 новых инструментов (Version Compatibility, Add Superclass Constructor Call и Optimize Imports) и исправлению 10 проблем (ложно-положительные/отрицательные срабатывания и др.)